

Laboratório — Semana 3

Arquitetura em Camadas na Prática: Controller, Service, Repository e Model

Disciplina: Desenvolvimento de Sistemas Corporativos **Curso:** Curso Superior de Tecnologia em Sistemas para Internet **Stack:** Java 17+ com Spring Boot 4 **Duração estimada:** 4 aulas (45 min cada) — quarta e sexta-feira

1. Objetivo do laboratório

Ao final deste laboratório, o estudante será capaz de:

- Estruturar um projeto Spring Boot seguindo a arquitetura em camadas (**Model, Repository, Service, Controller**);
- Explicar, com evidências no console, como uma requisição HTTP percorre cada camada até a resposta ser devolvida ao cliente;
- Justificar por que uma regra de negócio deve viver na camada de **Service**, e não no **Controller** nem no **Repository**;
- Testar uma API REST utilizando o navegador, `curl` ou uma ferramenta como Postman/Insomnia.

Observação importante: neste laboratório o **Repository** não acessa um banco de dados real — ele simula o armazenamento com uma lista em memória. A persistência com banco de dados real (Spring Data JPA) será tema da Semana 5. O foco aqui é exclusivamente a **organização em camadas**.

2. Pré-requisitos

- JDK 17 ou superior instalado (`java -version`);
 - IDE de preferência (IntelliJ IDEA, Eclipse ou VS Code com extensões Java);
 - Acesso à internet para gerar o projeto em start.spring.io;
 - Postman, Insomnia ou terminal com `curl` para testar a API.
-

3. Parte 1 — Gerando o projeto (≈10 min)

1. Acesse start.spring.io;
2. Configure o projeto com as seguintes opções:

Campo	Valor
Project	Maven
Language	Java
Spring Boot	versão estável mais recente da série 4.x
Group	<code>br.edu.ifrn</code>

Campo	Valor
Artifact	lab-tarefas
Packaging	Jar
Java	17 (ou superior)

3. Em **Dependencies**, adicione apenas:

- **Spring Web**
- **Spring Boot DevTools** (opcional, mas recomendado — reinicia a aplicação automaticamente a cada alteração)

4. Clique em **Generate**, extraia o **.zip** e importe o projeto na sua IDE.

5. Confirme que o projeto sobe corretamente executando a classe principal (**LabTarefasApplication**) e acessando **http://localhost:8080** (uma página de erro "Whitelabel" é esperada — ainda não criamos nenhum endpoint).

4. Parte 2 — Criando a camada de Dados (Model + Repository) (≈15 min)

4.1. Model — Tarefa

Crie o pacote **br.edu.ifrn.labtarefas.model** e, dentro dele, a classe **Tarefa**:

```
package br.edu.ifrn.labtarefas.model;

public class Tarefa {

    private Long id;
    private String titulo;
    private boolean concluida;

    public Tarefa(Long id, String titulo, boolean concluida) {
        this.id = id;
        this.titulo = titulo;
        this.concluida = concluida;
    }

    public Long getId() {
        return id;
    }

    public String getTitulo() {
        return titulo;
    }

    public boolean isConcluida() {
        return concluida;
    }
}
```

```
    public void setConcluida(boolean concluida) {  
        this.concluida = concluida;  
    }  
}
```

4.2. Repository — TarefaRepository

Crie o pacote `br.edu.ifrn.labtarefas.repository` e, dentro dele, a classe `TarefaRepository`:

```
package br.edu.ifrn.labtarefas.repository;  
  
import br.edu.ifrn.labtarefas.model.Tarefa;  
import org.springframework.stereotype.Repository;  
  
import java.util.*;  
import java.util.concurrent.atomic.AtomicLong;  
  
@Repository  
public class TarefaRepository {  
  
    private final Map<Long, Tarefa> banco = new LinkedHashMap<>();  
    private final AtomicLong sequencia = new AtomicLong();  
  
    public Tarefa salvar(String titulo) {  
        System.out.println("[REPOSITORY] Salvando tarefa em memória: " +  
titulo);  
        Long id = sequencia.incrementAndGet();  
        Tarefa tarefa = new Tarefa(id, titulo, false);  
        banco.put(id, tarefa);  
        return tarefa;  
    }  
  
    public List<Tarefa> listarTodas() {  
        System.out.println("[REPOSITORY] Buscando todas as tarefas em  
memória");  
        return new ArrayList<>(banco.values());  
    }  
  
    public Optional<Tarefa> buscarPorId(Long id) {  
        System.out.println("[REPOSITORY] Buscando tarefa por id: " + id);  
        return Optional.ofNullable(banco.get(id));  
    }  
}
```

Observação: usamos `System.out.println` apenas para deixar o fluxo entre camadas bem visível no console durante este laboratório introdutório. Em projetos reais, o recomendado é usar um logger (SLF4J, já incluído automaticamente pelo Spring Boot) no lugar de `System.out.println` — assunto que retomaremos em aulas futuras.

Pergunta de reflexão 1: por que a anotação `@Repository` está na classe, mesmo sem haver nenhum banco de dados real por trás dela? O que essa anotação sinaliza para o Spring e para quem lê o código?

5. Parte 3 — Criando a camada de Negócio (Service) (≈10 min)

Crie o pacote `br.edu.ifrn.labtarefas.service` e, dentro dele, a classe `TarefaService`:

```
package br.edu.ifrn.labtarefas.service;

import br.edu.ifrn.labtarefas.model.Tarefa;
import br.edu.ifrn.labtarefas.repository.TarefaRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class TarefaService {

    private final TarefaRepository repository;

    public TarefaService(TarefaRepository repository) {
        this.repository = repository;
    }

    public Tarefa criar(String titulo) {
        System.out.println("[SERVICE] Validando regra de negócio para: " +
            titulo);

        if (titulo == null || titulo.isBlank()) {
            throw new IllegalArgumentException("O título da tarefa não pode
            ser vazio.");
        }

        return repository.salvar(titulo.trim());
    }

    public List<Tarefa> listar() {
        System.out.println("[SERVICE] Solicitando lista de tarefas ao
        repository");
        return repository.listarTodas();
    }

    public Tarefa buscarPorId(Long id) {
        System.out.println("[SERVICE] Processando busca por id: " + id);
        return repository.buscarPorId(id)
            .orElseThrow(() -> new IllegalArgumentException("Tarefa não
            encontrada: " + id));
    }
}
```

Note que a regra **"título não pode ser vazio"** está no **Service**, não no **Controller** e não no **Repository**. Essa é a regra de negócio central desta atividade.

6. Parte 4 — Criando a camada de Apresentação (Controller) (≈10 min)

Crie o pacote `br.edu.ifrn.labtarefas.controller` e, dentro dele, a classe `TarefaController`:

```
package br.edu.ifrn.labtarefas.controller;

import br.edu.ifrn.labtarefas.model.Tarefa;
import br.edu.ifrn.labtarefas.service.TarefaService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/tarefas")
public class TarefaController {

    private final TarefaService service;

    public TarefaController(TarefaService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<Tarefa> criar(@RequestBody Map<String, String> corpo) {
        System.out.println("[CONTROLLER] Requisição recebida: POST /tarefas");
        Tarefa tarefa = service.criar(corpo.get("titulo"));
        return ResponseEntity.ok(tarefa);
    }

    @GetMapping
    public ResponseEntity<List<Tarefa>> listar() {
        System.out.println("[CONTROLLER] Requisição recebida: GET /tarefas");
        return ResponseEntity.ok(service.listar());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Tarefa> buscar(@PathVariable Long id) {
        System.out.println("[CONTROLLER] Requisição recebida: GET /tarefas/" + id);
        return ResponseEntity.ok(service.buscarPorId(id));
    }
}
```

Execute a aplicação novamente. Se tudo estiver correto, o console voltará a exibir apenas o log padrão do Spring Boot, sem erros de compilação.

7. Parte 5 — Testando e observando o fluxo (≈10 min)

7.1. Criando uma tarefa

Usando `curl` (terminal) ou Postman/Insomnia, envie:

```
curl -X POST http://localhost:8080/tarefas \
  -H "Content-Type: application/json" \
  -d "{\"titulo\": \"Estudar arquitetura em camadas\"}"
```

Observe o console da aplicação. Você deverá ver, na ordem, os logs:

```
[CONTROLLER] Requisição recebida: POST /tarefas
[SERVICE] Validando regra de negócio para: Estudar arquitetura em camadas
[REPOSITORY] Salvando tarefa em memória: Estudar arquitetura em camadas
```

Essa sequência é a prova visual de que a requisição atravessou **Controller → Service → Repository**, nessa ordem, e não de outra forma.

7.2. Listando as tarefas

```
curl http://localhost:8080/tarefas
```

7.3. Testando a regra de negócio

Envie agora uma tarefa com título vazio:

```
curl -X POST http://localhost:8080/tarefas \
  -H "Content-Type: application/json" \
  -d "{\"titulo\": \"\"}"
```

Você deverá receber um erro (HTTP 500, por ora — trataremos exceções de forma adequada com `@ExceptionHandler` na Semana 11). O importante aqui é **onde** o erro foi lançado: no log, `[SERVICE]` aparece, mas a linha do `[REPOSITORY]` **não aparece** — prova de que a validação barrou a requisição antes de chegar à camada de dados.

Pergunta de reflexão 2: o que aconteceria se essa mesma validação estivesse escrita dentro do `TarefaController`, em vez do `TarefaService`? Tecnicamente funcionaria — então por que essa não é a prática recomendada?

8. Desafio

Adicione um novo endpoint **PUT /tarefas/{id}/concluir**, que marca uma tarefa como concluída. Siga rigorosamente o fluxo em camadas:

- O **Controller** recebe o **id** e delega ao Service — não altera o objeto **Tarefa** diretamente;
- O **Service** busca a tarefa (reaproveitando **buscarPorId**), aplica a regra **tarefa.setConcluida(true)** e poderia, futuramente, aplicar outras regras (ex.: impedir concluir uma tarefa já concluída);
- O **Repository** não precisa de um novo método, pois a lista já guarda a referência ao mesmo objeto em memória.

Pergunta de reflexão 3 (desafio): ao adicionar essa nova funcionalidade, você precisou alterar a camada de Dados? Isso é esperado? O que isso demonstra sobre os benefícios da separação em camadas?

9. Roteiro de entrega

Ao final do laboratório, cada dupla deve entregar (conforme orientação do professor: impresso, por link de repositório Git, ou apresentação oral rápida):

1. O código-fonte do projeto **lab-tarefas**, com as quatro camadas implementadas;
2. Um print (ou trecho colado) do console mostrando a sequência de logs **[CONTROLLER] → [SERVICE] → [REPOSITORY]** para uma requisição de criação de tarefa;
3. As respostas escritas às **três perguntas de reflexão** deste roteiro.

10. Critérios de avaliação (formativa, não substitui a Avaliação 1)

Critério	O que será observado
Organização em pacotes	model , repository , service e controller corretamente separados
Uso correto das anotações	@RestController , @Service , @Repository aplicadas na camada correspondente
Injeção de dependência	Uso de construtor para injetar dependências (sem new manual das dependências)
Regra de negócio no lugar certo	Validação implementada no Service, não no Controller
Reflexões	Respostas demonstrando compreensão do fluxo entre camadas

11. Próximos passos

Na **Semana 4**, avançaremos para os componentes da lógica de negócio (**@Service**, DTOs e padrões de projeto como Strategy e Factory) e iniciaremos, em equipes, a estruturação do projeto corporativo final —

que deverá seguir exatamente esta mesma organização em camadas, agora com Spring Data JPA para persistência real, a partir da Semana 5.